

南京信息工程大学

《数据结构》课程设计

姓名 张正珂

学号 202483290476

专业 计算机科学与技术

二〇二五年 12 月 28 日

《数据结构》课程设计

张正珂

计算机学院 计算机科学与技术 2024 级 6 班

摘要：本报告详细阐述了三个数据结构课程设计任务的实现过程与结果。首先，利用栈结构实现了括号匹配的检验算法，能够有效识别嵌套括号序列的合法性；其次，基于哈夫曼树构建了编码与译码系统，实现了对文本数据的高效压缩与还原；最后，实现了十种常见的内部排序算法，并通过大量随机数据对比分析了它们在比较次数与移动次数上的性能差异。

关键词：栈；哈夫曼树；内部排序；算法分析

第一章 链表的应用（或栈和队列的应用）

1.1 问题描述

假设表达式中允许有两种括号：圆括号和方括号，其嵌套的顺序随意，即 $(()[])$ 或 $[([[]])]$ 等为正确格式， $[()]$ 或 $(([])$ 均为不正确的格式。本任务要求读入圆括号和方括号的任意序列，输出“匹配”或“此串括号匹配不合法”。

1.2 算法思想

该算法利用“栈”（Stack）这种后进先出（LIFO）的数据结构来处理嵌套关系。

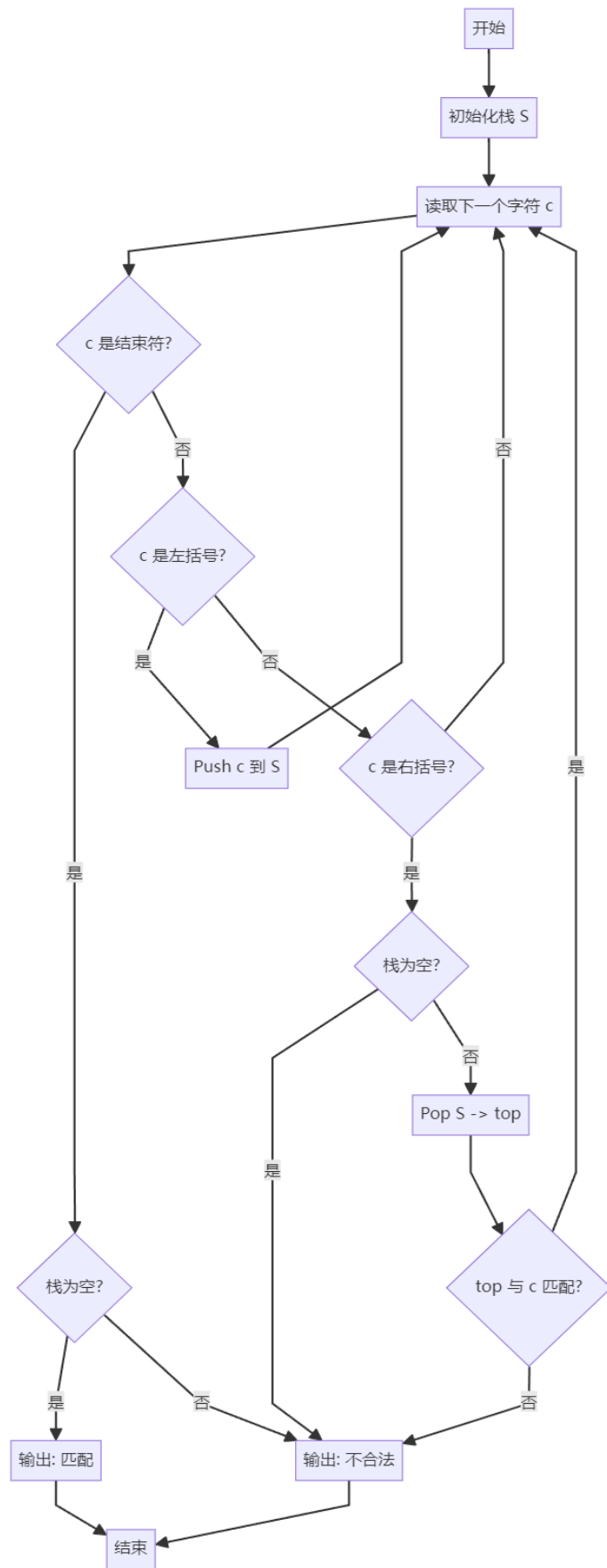
- 初始化：创建一个空栈。
- 遍历输入字符串：逐个读取字符。
- 左括号处理：遇到 $($ 、 $[$ 或 $\{$ 时，将其压入栈中。
- 右括号处理：遇到 $)$ 、 $]$ 或 $\}$ 时：

若栈为空，说明右括号多余，序列不合法。

若栈不为空，弹出栈顶元素。检查弹出的左括号与当前右括号是否匹配（如 $($ 对应 $)$ ）。若不匹配，则序列不合法。

- 结果判断：遍历结束后，若栈为空，说明所有左括号都已匹配，序列合法；否则（栈内仍有残留左括号），序列不合法。

流程图：

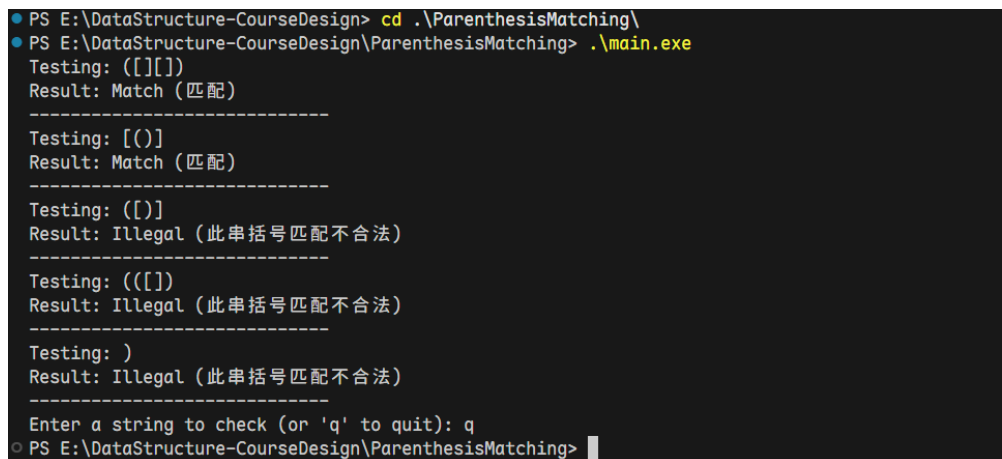


(图 1.1)

1.3 算法代码

```
bool Match(char left, char right) {  
    if (left == '(' && right == ')') return true;  
    if (left == '[' && right == ']') return true;  
    if (left == '{' && right == '}') return true;  
    return false;  
}  
  
void CheckParenthesis(const string& str) {  
    stack<char> s;  
    bool valid = true;  
    // ... (遍历字符串逻辑) ...  
    if (valid && s.empty()) {  
        cout << "Result: Match (匹配)" << endl;  
    } else {  
        cout << "Result: Illegal (此串括号匹配不合法)" << endl;  
    }  
}
```

1.4 运行结果



```
PS E:\DataStructure-CourseDesign> cd .\ParenthesisMatching\  
PS E:\DataStructure-CourseDesign\ParenthesisMatching> .\main.exe  
Testing: ([][])  
Result: Match (匹配)  
-----  
Testing: [(())]  
Result: Match (匹配)  
-----  
Testing: ([)]  
Result: Illegal (此串括号匹配不合法)  
-----  
Testing: ((()))  
Result: Illegal (此串括号匹配不合法)  
-----  
Testing: )]  
Result: Illegal (此串括号匹配不合法)  
-----  
Enter a string to check (or 'q' to quit): q  
PS E:\DataStructure-CourseDesign\ParenthesisMatching>
```

(图 1.2)

第二章 树结构的应用（或图结构应用）

2.1 问题描述

设计一个利用哈夫曼算法进行编码和译码的系统。

初始化 (I)：输入字符集和权值，建立哈夫曼树。

编码 (E)：对输入文本进行编码，结果存入文件。

译码 (D)：对代码文件进行译码，还原为文本。

打印 (P/T)：显示代码文件内容和哈夫曼树结构。

2.2 算法思想

1. 构建哈夫曼树：

根据给定的 n 个权值构造 n 棵只有根结点的二叉树。

在森林中选取两棵根结点权值最小的树作为左右子树构造一棵新二叉树，新树根权值为左右子树根权值之和。

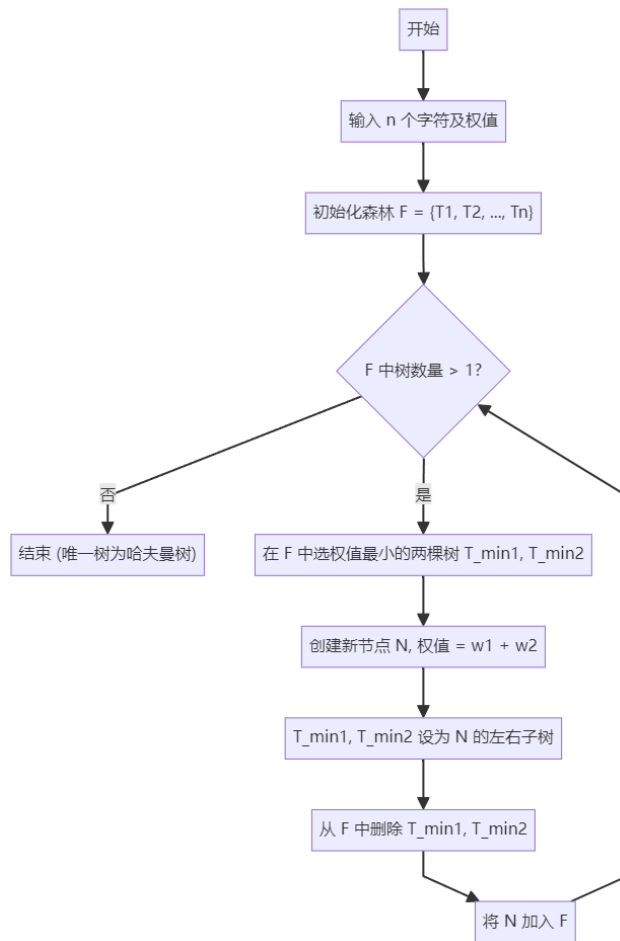
删除选中的两棵树，将新树加入森林。

重复上述步骤直到只剩一棵树。

2. **编码：**从叶子结点向上回溯到根，左分支记为 '0'，右分支记为 '1'，得到的序列逆序即为编码。

3. **译码：**从根结点出发，读取编码串，'0' 走左孩子，'1' 走右孩子，走到叶子结点即译出一个字符，然后回到根结点继续。

流程图（构建哈夫曼树）：



(图 2.1)

2.3 算法代码

```
void Select(int k, int& s1, int& s2) {  
    // 寻找权值最小的两个根结点 s1, s2  
    // ...  
}  
  
// 初始化与建树  
for (int i = n + 1; i <= m; ++i) {  
    int s1, s2;  
    Select(i - 1, s1, s2);  
    HT[s1].parent = i; HT[s2].parent = i;  
    HT[i].lchild = s1; HT[i].rchild = s2;  
    HT[i].weight = HT[s1].weight + HT[s2].weight;  
}
```

2.4 运行结果

```
PS E:\DataStructure-CourseDesign\HuffmanCoding> .\main.exe  
  
-----  
      Huffman Coding System  
-----  
I: Initialization  
E: Encoding  
D: Decoding  
P: Print Code File  
T: Print Huffman Tree  
Q: Quit  
Enter choice: i  
Enter number of characters (n): 52  
Enter 52 characters and their weights (e.g., 'a 5'):  
A 1  
B 1  
C 1  
D 1  
E 1  
F 1  
G 1  
H 1  
I 1 J 1 K 1 L 1 M 1 N 1 O 1 P 1 Q 1 R 1 S 1 T 1 U 1 V 1 W 1 X 1 Y 1 Z 1  
a 1 b 1 c 1 d 1 e 1 f 1 g 1 h 1 i 1 j 1 k 1 l 1 m 1 n 1 o 1 p 1 q 1 r 1 s 1 t 1 u 1 v 1 w 1 x  
1 y 1 z 1  
Huffman Tree saved to hfmTree.txt
```

(图 2.2) 初始化字符权重

```
CodeFile Content:
101110111010111100111010000011011010101111010110
01011011100011110110110110100000111111001011101100
0011111111101100010111001111001000100110110110101
110110111111101000101011010011110100000000111011
01101011110101100100010000101110010000110010111011
011010111101011111001011110011100100010100000000
1110110000111100100010111011000100001101111111010
111011000011110010111111000000001110110111110010
0000000011000011110100110110111110110010111111100
10111000110110111110110110111111001010010000101010
101111011101101000000010111101000100000000111101
1000011001001101101101111111001101010001011110
011101101111100000111110000100011000001101111110
1000101101110111011100111001110101111101101101011
1101011001001110100000001101111111010111001000101
1110100000011111101110111010111100111010000011101
10110101111010110010001100010011011000100110100000
00111101111101110010110011000000001111001000101111
01000111110110001000000011011100101010001100100001
1110110111100111110000001000111111110010001001100
10010001110101111001110100010111100111001000101110
11111001011010011101011110111101000101110010001011
10110001000010111100111011011010000011110110110010
001011110100000011111110010111111101011101011101
1000111101101111010000000011111101101111111001
01000001101111100100001100101111010110100111101110
1100010001100011110100101111001000000110111000111
00100100000001111101101110011110101110001110011
11101010100000100110110011111101111010110100111010
01011110110110101000010001100000110011111101110110
11111000100111001110010001111101101101001100101111
0111110111011011010111001000101001011101101111100
10111101000000111111001010000010111011101011110011
10100000111011011010111101011001000100110110110101
1110100010100000000111101011001011110100001000110
00001101001101100010000100001011110011101100101011
1001110010001111101101101010000011111110110111101
11101000101001011111011101100010100000110101110010
111111000001110110111111000010011011001111110111101
01101000011000100110110000001101110010111100111011
00001111011000100000000011000011110100110110010001
1100111101011010011100111101000100000001111111011
00000011011100101111001110110111110000000010000101
00111111010001001110100010111011011010100100111010
00101110110001000000011111100101111001110110100000
11111110010111011000011111111101100010110011000011
11101011100011101011111111001011111111010111100000
01100000010000010111110011000001111110000111101011
11101111100101010111011001011110111011000100110010
0010000110110100110100110110001000100110111001101
```

(图 2.3) 哈夫曼编码后(部分)

```
HuffmanCoding > TextFile.txt
1  WikipediapfreeInternetbasedencyclopediastartedinthatoperatesunderanopensourcemangementstyleItisoverseen
```

(图 2.4) 哈夫曼解码后(部分)

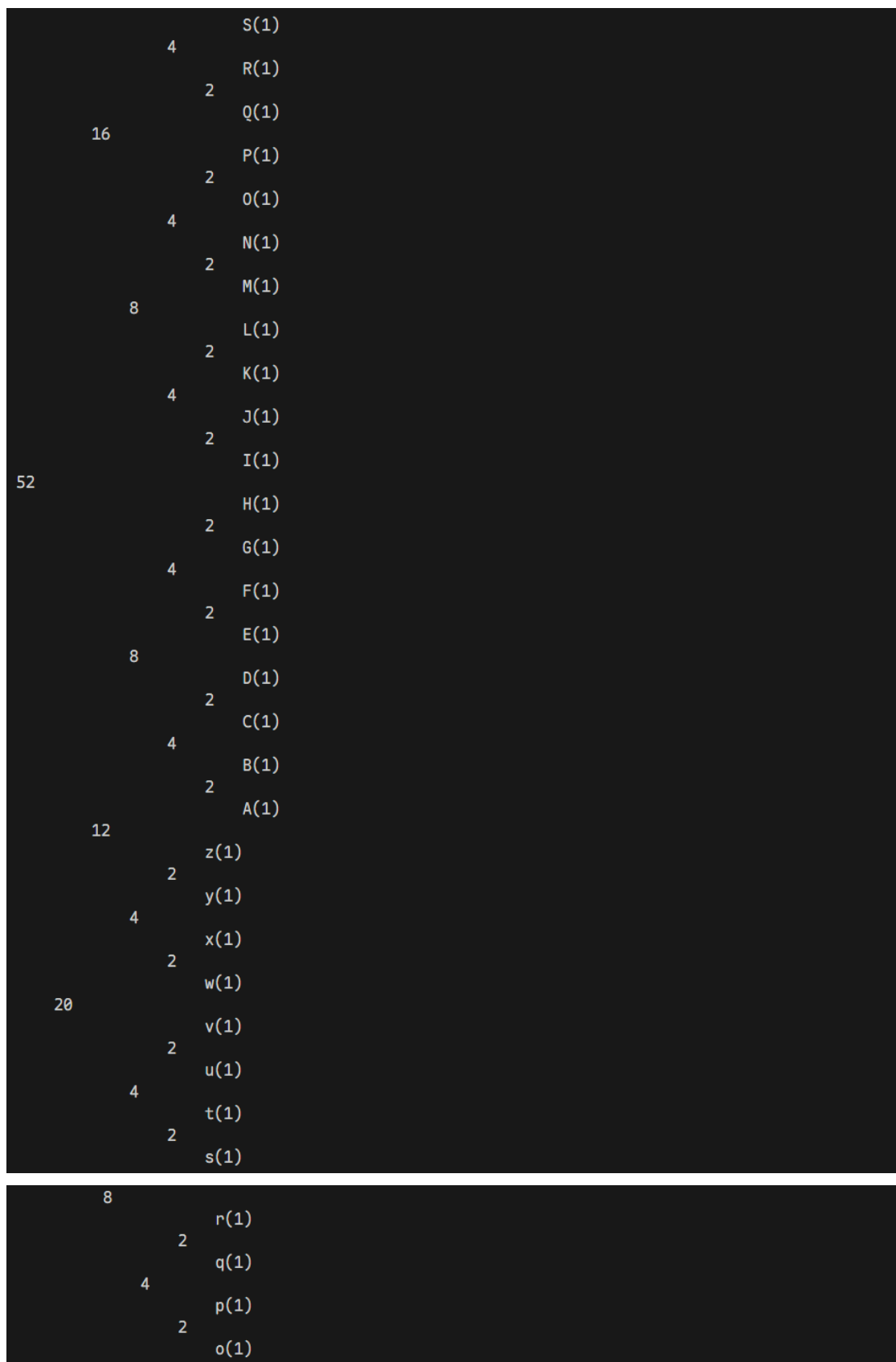
Huffman Coding System

I: Initialization
E: Encoding
D: Decoding
P: Print Code File
T: Print Huffman Tree
Q: Quit
Enter choice: t

Huffman Tree (Right-rotated 90 degrees):

```

      n(1)
      2
      m(1)
    4
    l(1)
    2
    k(1)
  8
  j(1)
  2
  i(1)
  4
  h(1)
  2
  g(1)
16
  f(1)
  2
  e(1)
  4
  d(1)
  2
  c(1)
  8
  b(1)
  2
  a(1)
  4
  Z(1)
  2
  Y(1)
32
  X(1)
  2
  W(1)
  4
  V(1)
  2
  U(1)
  8
```

(图 2.5) 哈夫曼树

第三章 小型信息管理系统设计

3.1 问题描述

对 10 种常用的内部排序算法（直接插入、折半插入、二路插入、希尔、起泡、快速、简单选择、堆、归并、基数排序）进行比较。

要求：待排序表长不少于 100，使用随机数据，至少比较 5 组不同数据，统计比较次数和移动次数。

3.2 算法思想

系统通过随机数生成器产生不同规模（如 100, 200, ..., 2000）的整数序列。针对每种算法：

1. **数据加载**：将随机序列复制到排序对象中。
2. **执行排序**：调用特定算法进行排序。在关键比较操作（LT, GT 等）和移动操作（赋值、交换）处插入计数器。
3. **统计输出**：排序完成后，输出该次运行的比较次数与移动次数。

主要算法简述：

快速排序：采用分治法，选取枢轴将表分为两部分，左边小于枢轴，右边大于枢轴，递归处理。

堆排序：利用堆的性质（大顶堆），将堆顶元素与末尾元素交换，调整堆，重复直到有序。

希尔排序：缩小增量排序，先将序列按增量分组进行直接插入排序，增量逐渐减小至 1。

3.3 算法代码

```
struct SortStats {
    long long comparisons;
    long long moves;
    void reset() { comparisons = 0; moves = 0; }
};

// 示例：直接插入排序
void InsertSort() {
    for (int i = 2; i <= length; ++i) {
        if (LT(r[i].key, r[i - 1].key)) { // 比较计数
            r[0] = r[i];
```

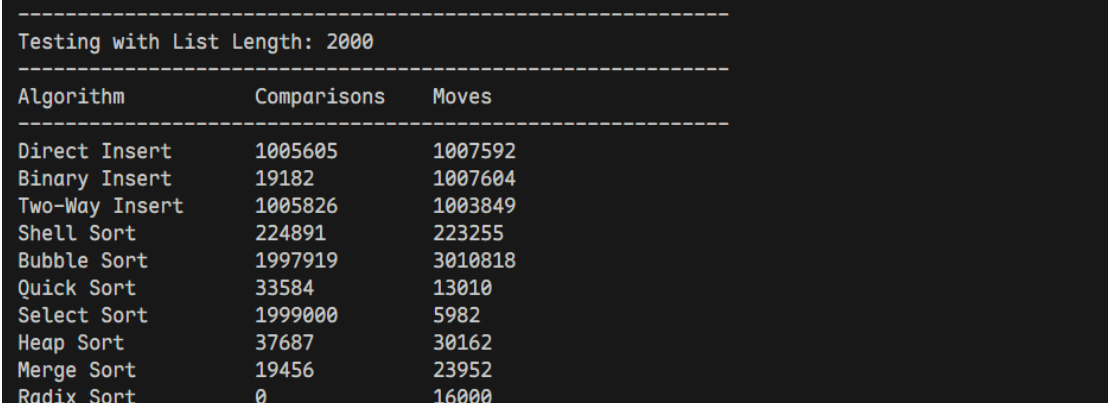
```

        stats.moves++; // 移动计数
        // ... 移动与插入 ...
    }
}
}

```

运行结果

测试数据长度为 100, 200, 500, 1000, 2000。以下是部分（长度 2000）的典型运行结果截图数据：



Testing with List Length: 2000		
Algorithm	Comparisons	Moves
Direct Insert	1005605	1007592
Binary Insert	19182	1007604
Two-Way Insert	1005826	1003849
Shell Sort	224891	223255
Bubble Sort	1997919	3010818
Quick Sort	33584	13010
Select Sort	1999000	5982
Heap Sort	37687	30162
Merge Sort	19456	23952
Radix Sort	0	16000

(图 3. 1)

分析：

时间复杂度：快速排序、堆排序、归并排序表现优秀，比较次数远少于 $O(n^2)$ 的冒泡和直接插入排序。

移动次数：基数排序移动次数最少且稳定（与位数有关），链式基数排序会更优。

稳定性：直接插入、冒泡、归并、基数排序是稳定的；希尔、快速、堆排序是不稳定的。

第四章 总结

本次课程设计完成了括号匹配、哈夫曼编/译码以及内部排序算法比较三个项目。

1. 工作内容：

深入理解了栈、二叉树（哈夫曼树）以及各种排序算法的原理。

使用 C++ 实现了上述数据结构与算法，并设计了相应的测试用例与统计模块。

学会了如何通过计数器量化分析算法性能。

2. 特色与不足：

特色：排序系统实现了全自动化的多组数据对比，能够直观展示不同算法随数据

规模增长的性能变化趋势；哈夫曼系统界面友好，功能完备。

不足：哈夫曼系统的字符集目前仅限于 ASCII 字符，对于中文等宽字节字符支持尚需改进；排序算法演示部分若能增加可视化动画会更直观。

3. **收获：**通过实践，不仅巩固了理论知识，更提高了编程调试能力和算法分析能力，深刻体会到了数据结构在解决实际问题中的核心作用。

参考文献：

- [1] 李冬梅，严蔚敏，吴伟民. 《数据结构（C语言版）》. 清华大学出版社. 2007.
- [2] 王红梅. 《数据结构（C++版）》. 清华大学出版社. 2011.
- [3] Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to Algorithms. MIT Press. 2009.